



Custom Systolic-Array Accelerator for RAFT Convolution

Milestone 4 – Design Justification Report

ECE 510: Hardware for AI and Machine Learning

Elliott Day

Portland State University

Spring 2026 | June 7, 2026

16×16 weight-stationary systolic array, bf16×bf16→fp32, 100 MHz, sky130 (OpenLane 2).

How to use this template (delete before final). Replace every **[red blank]** and answer every **QUESTION FOR AUTHOR** box, then remove this box and the list below. Keep the nine section headings exactly as-is (the grader counts them). Aim for 2000–2500 words total. Build with `make`.

QUESTION FOR AUTHOR: Optional polish only: all required sections are filled (name, precision, diagrams, verification, synthesis, benchmark, what-did-not-work). Add anything in Sec. 1 about the end application you want to emphasize, then delete the boxes above before submitting.

Abstract

This report documents a custom hardware accelerator for the dominant convolution kernel of the RAFT `raft_large` optical-flow network. The design is a 16×16 weight-stationary systolic array (256 multiply-accumulate cells) using bf16 multiplication with fp32 accumulation, wrapped in an AXI4-Stream data plane and an AXI4-Lite control plane. It is verified in cocotb co-simulation and pushed through the OpenLane 2 / sky130 flow, closing timing at 100 MHz (worst setup slack +0.667 ns) with an estimated 0.834 W and 16.38 mm² die. Benchmarked against the M1 CPU software baseline on the target layer, the accelerator is currently memory/reload bound (effective arithmetic intensity 0.92 FLOP/byte) and does *not* beat the CPU on this layer. The shortfall is a planning gap – a per-tile weight-reload schedule that I did not integration-test until late – not a datapath defect; this report analyzes the root cause, the streaming fix I prototyped, and why it was deferred.

1 Problem and motivation

This project accelerates RAFT (Recurrent All-Pairs Field Transforms), a learned optical-flow network, with the target application being machine-learning-based SLAM front-ends for edge robotics. Learned-SLAM front-ends built on RAFT-style flow and correspondence are accurate but, on a GPU, power-hungry and physically bulky; while GPUs and accelerators exist for related learned-SLAM kernels, there is no RAFT-specific hardware. The goal is to run the same workload inside an edge power/area budget at real-time rates (30–60 fps), trading the GPU’s generality for efficiency on this one network.

Two properties make RAFT a good custom-silicon target. First, it is **convolution-dominated**: M1 profiling of `raft_large` (project/m1/sw_baseline.md, codefest/cf02/profiling) found 2D convolution (`aten::mkldnn_convolution`) to be the dominant cost at **53.02%** of forward self-CPU time, on a mean forward of **4.145 s** and **788.14 GMAC** per forward. Second, it **tolerates reduced precision**: the M2 analysis (Sec. 3) shows bf16 multiply with fp32 accumulate meets an engineering error bound, so the area-dominant multipliers can be small. A weight-stationary systolic array exploits convolution’s regular multiply-accumulate structure and weight reuse directly in silicon, motivating a custom datapath rather than a general-purpose core.

The original proposal targeted a 48×48 array; the delivered scope is a 16×16 array at 100 MHz (Sec. 7 explains the scope reduction). The rest of this report characterizes that design and is candid about where it falls short of the real-time goal (Sec. 8–9).

2 Roofline analysis

The target layer (Conv2d 5-1, $C_{in}=C_{out}=64$, 3×3 , batch 4, 260×480) has ~ 36.9 GFLOP. With fp32 operands and no reuse its intrinsic arithmetic intensity is ~ 144 FLOP/byte. The accelerator’s

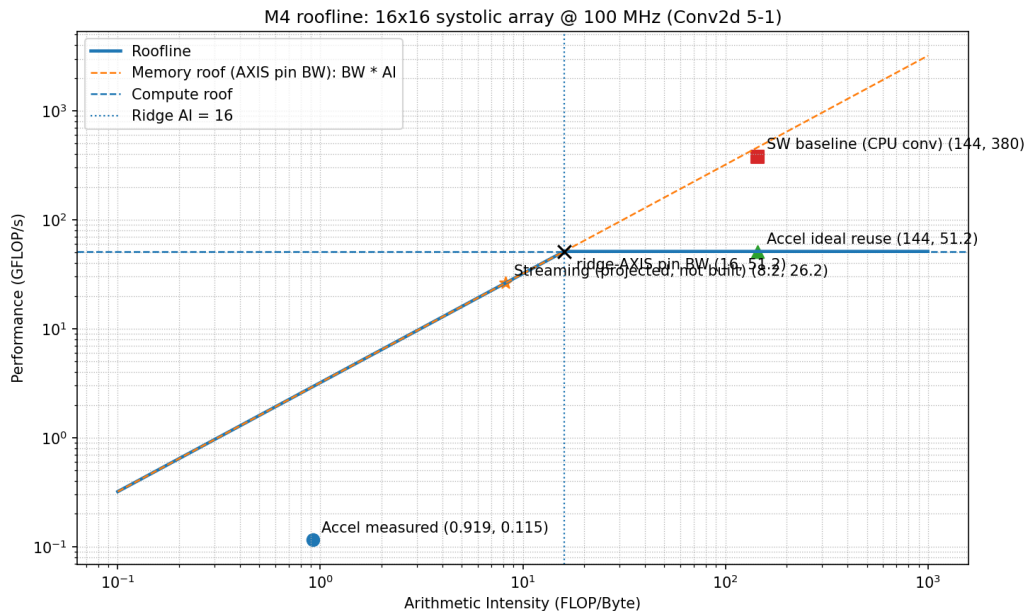


Figure 1: Roofline for the 16×16 array (compute roof 51.2 GFLOP/s, pin bandwidth 3.2 GB/s, ridge AI = 16). The measured operating point sits at AI 0.92, 0.115 GFLOP/s; the CPU software baseline and the (not-built) streaming projection are shown for context.

compute roof is $256 \times 2 \times 100 \text{ MHz} = 51.2 \text{ GFLOP/s}$; its binding memory roof is the AXI4-Stream pin bandwidth (one 256-bit beat/cycle at 100 MHz = 3.2 GB/s), giving a ridge point at $\text{AI}^* = 16 \text{ FLOP/byte}$. As built, the schedule reloads the weight slab on every K-tile and reuses resident weights across zero extra pixels, so the *measured* effective AI collapses to **0.92 FLOP/byte** – deep in the memory/reload-bound region (Fig. 1).

The design is memory-bottlenecked in a way I did not anticipate, and the cause is a planning gap rather than a single bad gate. Because weights are reloaded per tile, the weight-streaming cost grows steeply with the number of pixels that must be convolved, and that weight movement – not the 256 MAC/cycle the array can sustain – ends up dominating runtime. The two fixes I identified are to enlarge the on-chip weight store and to stream a block of pixel-columns through each resident weight set so one load is amortized over many pixels (Sec. 4); I attempted the streaming version but hit further LibreLane/PnR issues (Sec. 9) and shipped this flawed-but-closing design instead. The central architectural lesson stands: the dataflow’s reuse, not the array’s peak, sets delivered performance.

3 Precision and data format

The datapath multiplies two bf16 operands, accumulates in fp32 across the K-reduction, and drains the result back to bf16 (round-toward-zero) on output; subnormal inputs flush to signed zero. The product of two normal bf16 values is exact in the fp32 mantissa, so the only rounding sources are the fp32 adds and the final bf16 drain. bf16 keeps the 8-bit exponent of fp32 (same dynamic range) while shrinking the multiplier mantissa to 8×8 – the dominant area term – and fp32 accumulation avoids the fp16-style creep that would otherwise build over the long convolution inner products.

I validated this in M2 (`project/m2/precision.md`) with a Gaussian stress sweep (`tb/tb_quant_error.py`):

top.sv block diagram -- 16x16 systolic accelerator (AXI4-Stream data plane + AXI4-Lite control plane)

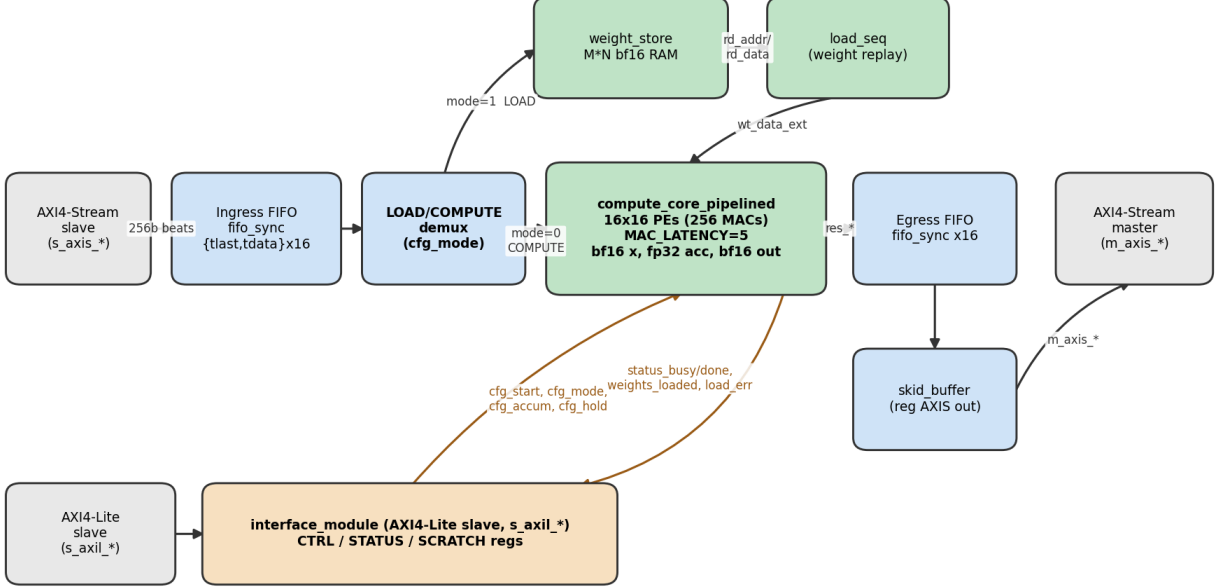


Figure 2: top.sv block diagram: AXI4-Stream data plane (ingress FIFO, LOAD/COMPUTE demux, weight_store + load_seq, compute core, egress FIFO, skid buffer) and AXI4-Lite control plane.

SEED=42, 30 tiles $\rightarrow$ **1440** bf16 outputs, four times the 100-sample rubric minimum, run on the headline 48×48 array so the reduction depth ($K=48$) is deeper than – and bounds – the shipped 16×16 tile. The hardware was compared against a software model that round-trips through fp32 after every multiply and add (tracking fp32, not fp64). Measured error: **MAE** $= 4.87 \times 10^{-2}$, RMSE 6.28×10^{-2} , max abs error 2.70×10^{-1} . The dominant error bin $[10^{-2}, 10^{-1})$ is the expected $\sqrt{K}$ growth as reduction depth rises, and the dimensionless $\text{MAE}/\sqrt{\text{Var}(y)}$ stays small.

I declare an **engineering pass** when $\text{MAE} < 10^{-1}$ against this stimulus; the measured 4.87×10^{-2} clears it with $\sim 2\times$ margin. The acceptability ceiling is tied to the end task: RAFT reports KITTI/Sintel end-point error at the sub-pixel to few-pixel scale, so feature noise at $\text{MAE} < 5 \times 10^{-2}$ is small relative to the iterative refinement and motion magnitudes that dominate the benchmark. (Round-toward-zero adds a small one-sided bias on long dot products; switching the accumulate/drain to round-to-nearest-even is the documented precision follow-up.)

4 Dataflow and architecture

The top-level structure (Fig. 2) is a bus-pin-only wrapper: an AXI4-Stream ingress FIFO feeds a one-bit **cfg_mode** demux that routes each beat either to **weight_store** (LOAD) or to the compute core (COMPUTE); **load_seq** replays the stored weights into the PE array; and results return through an egress FIFO and a registered **skid_buffer** to the AXI4-Stream master. An **interface_module** (AXI4-Lite) supplies **cfg_start**/mode/accum/hold and reports **status_busy/done**.

The array is **weight-stationary**: a 16×16 tile of weights is loaded into the 256 PEs and held while activations stream through, so each weight is read once per tile and reused across the activa-

Compute-core dataflow and scheduling -- compute_core_pipelined (M=N=16, MAC_LATENCY=5)

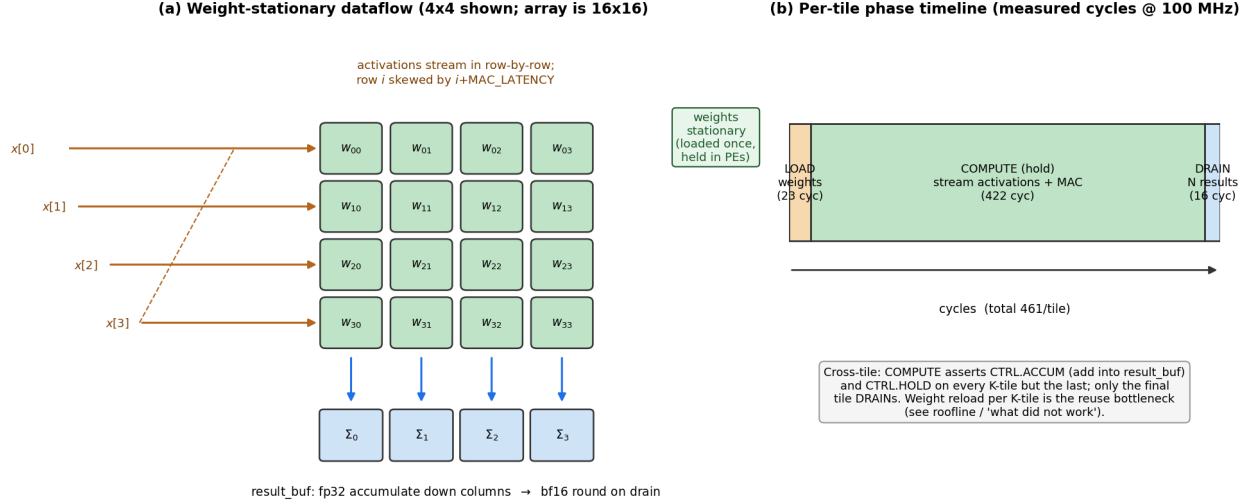


Figure 3: Compute-core dataflow. (a) Weights stay resident in the PE grid while activations enter row-by-row with the `MAC_LATENCY=5` skew and partial products accumulate down each column in fp32, rounding to bf16 only on drain. (b) The per-tile LOAD/COMPUTE(hold)/DRAIN phase timeline with measured cycle counts; cross-K tiles use `CTRL.ACCUM/HOLD` and only the last tile drains.

tion columns fed during that tile (Fig. 3). Each PE performs one bf16 multiply and one fp32 add per cycle (`pe_pipelined.sv`, `mul_bf16_p2`, `add_fp32_p2`); results reduce down each column into 16 fp32 accumulators (`result_buf`). A LOAD/COMPUTE/DRAIN FSM (`compute_core_pipelined.sv`) sequences weight load, activation streaming with the pipeline skew (`MAC_LATENCY`), and result drain; `weight_store` and `load_seq` stage the weight slab, and two synchronous FIFOs buffer the AXI-Stream ingress/egress. Weight-stationary fits convolution because the same filter weights apply across many output pixels – though, as Sec. 2 notes, the *current* tile schedule does not exploit that cross-pixel reuse.

The architectural weakness is here: the tile schedule reloads the full weight slab for every K-tile, so the weight-load cycles scale with the pixel count rather than amortizing across it, and weight streaming becomes the runtime bottleneck. Two RTL changes would close most of the gap. First, enlarging the `weight_store` so more of the filter bank stays resident reduces how often the slab is reloaded. Second – the larger win – streaming a block of B pixel-columns through each resident weight set amortizes one weight load over B pixels, raising effective AI by roughly the block factor. I prototyped the streaming path but could not get it through PnR in the time available (Sec. 9), so the shipped design keeps the simpler per-pixel schedule.

5 Hardware interface

The data plane is **AXI4-Stream** (256-bit `tdata`) and the control plane is **AXI4-Lite** (host writes mode/start, polls status). AXI4-Stream was chosen in M1 (`project/m1/interface_selection.md`) for high-throughput bulk tensor streaming. At 100 MHz the stream delivers 3.2 GB/s of pin bandwidth. The design is **interface/bandwidth bound** at the current operating point: at AI 0.92 FLOP/byte the 3.2 GB/s roof caps performance near 2.9 GFLOP/s, and the reload-stalled schedule

delivers even less (0.115 GFLOP/s, see Sec. 8). The array’s 51.2 GFLOP/s compute capability is therefore not the limiter – data movement is.

6 Verification

Correctness was verified by cocotb co-simulation (`tb/tb_top.py`), driving the DUT *only* through its AXI pins – no internal compute-core probes – and checking every output against an *independent* software reference (bf16-accurate or fp64), never a prior DUT run. The default run is **6 PASS + 2 skipped** (the two opt-in cases run under environment flags); `sim/cosim_run.log` and `tb/results.xml` record the contract:

`test_top_smoke` – after reset, all bus outputs sit at sane idle values.
`test_axil_scratch_loopback` – AXI4-Lite write/read of the SCRATCH register, proving the control path before anything depends on it.
`test_gemm_tile_e2e` (headline) – one im2col→GEMM tile of the M1 dominant kernel: load the $M \times N$ weight slab, run a COMPUTE tile, and check all N outputs bit-exact against the bf16 reference.
`test_weight_reuse_two_activation_tiles` – load weights once, run two activation tiles with no second weight push; a broken weight cache would stall and time out.
`test_backpressure` – hold `m_axis_tready=0` for 6 cycles after the first beat and confirm the remaining $N-1$ results still arrive bit-exact, exercising the AXIS handshake and the egress FIFO decoupling.
`test_conv_e2e` – end-to-end tiled convolution with *hardware* cross-tile fp32 accumulation (CTRL.ACCUM/HOLD), sweeping K in M -deep slices and comparing every output pixel/channel bit-exact to an fp64 convolution reference.
Opt-in: `test_conv_rafft_dims_e2e` (RUN_RAFT_DIMS, $C_{in}=C_{out}=64, 3 \times 3$) and `test_benchmark` (RUN_BENCH, measures the per-phase cycle counts used in Sec. 8).

Together these cover the control path, the AXIS handshake under backpressure, single-tile GEMM correctness (bf16 multiply / fp32 accumulate / bf16 round), weight-cache reuse, and multi-tile convolution accumulation. Figure 4 shows a representative co-simulation waveform.

7 Synthesis results

Synthesis and PnR used OpenLane 2 / sky130 (RUN_2026-06-04_17-46-11); numbers below are the post-global-route signoff snapshot (step 42, typical corner). The run entered detailed routing but did not finish it (see Sec. 9); step-42 STA is the timing snapshot.

Metric	Value	Dominant contributor
Clock / timing	100 MHz, setup MET +0.667 ns (0 setup vio)	reg-to-reg +1.518 ns
Hold	−0.969 ns WNS, 35 vio	IO-port paths only (reg-to-reg clean)
Die area	16.38 mm ² (std-cell 8.43 M μm ²)	combinational ~54%
Power (static est.)	0.834 W	sequential 48.8%, clock 36.6%

Table 1: Synthesis signoff summary (`synth/timing_report.txt`, `area_report.txt`, `power_report.txt`).

The dominant area contributor is combinational logic (~54% of placed std-cell area): the bf16

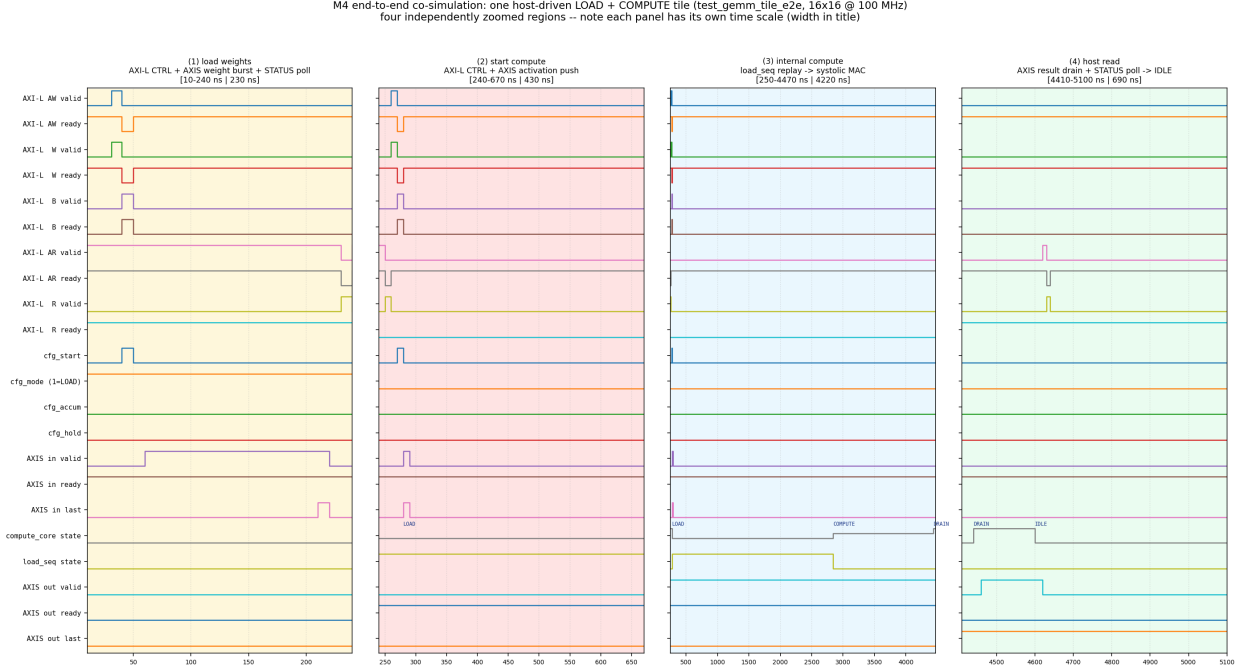


Figure 4: Co-simulation waveform of a GEMM tile through the AXI pins (`sim/render_waveform.py`): the LOAD/COMPUTE handshake, activation streaming, and result drain.

multipliers and fp32 adders replicated across 256 PEs. Power is dominated by the $\sim 96.6\text{k}$ async-reset flops (48.8%) and the clock network (36.6%), as expected for a flop-heavy systolic array. The power figure is OpenROAD’s *static* activity estimate, not VCD-back-annotated; the relative split (sequential $\gg$ clock $\gg$ combinational) is the reliable takeaway.

The 16×16 @ 100 MHz operating point is a deliberate scope choice. The array’s peak compute (51.2 GFLOP/s) sits below the CPU baseline largely because of those two numbers – the array dimensions and the clock. Earlier, larger and faster configurations turned every iteration into a 20-hour LibreLane run and left me debugging the flow instead of the design. Scaling back to 16×16 at 100 MHz let me actually close timing (Table 1) and iterate on the architecture. Within the constraints of this course project – limited tool time and a still-developing PnR workflow – I stand by that trade: a smaller design that closes and is fully characterized is more defensible than a larger one stuck mid-flow.

8 Benchmark results

Per-phase cycle costs were measured in simulation (`bench/bench_measured.csv`: load 23, compute-hold 422, drain 16 cycles) and extrapolated to the full layer by `bench/benchmark.py` (`benchmark_data.csv`). At 100 MHz the layer takes **320.2 s** (sustained **0.115 GFLOP/s**, 0.22% of the 51.2 GFLOP/s roof). The M1 CPU baseline runs the same layer in **0.097 s** (380 GFLOP/s effective conv rate), so the kernel speedup is **0.0003 $\times$** – the accelerator is roughly **3,300 $\times$ slower** on this layer, and slower on energy (267 J vs 4.36 J). This is reported honestly: the gap is architectural, not a clock or array-size issue. The CPU’s BLAS convolution keeps weights resident and reuses them across the whole output map; the array’s tile schedule reloads weights every K-tile and achieves no pixel reuse,

paying DMA and pipeline-fill cost that swamps the 256 MAC/cycle the array can sustain. The measured point even falls below the 3.2 GB/s bandwidth ceiling because of reload stalls (Fig. 1).

Why compute_hold scales so badly with pixels. The cost model is closed-form over the tile schedule the RTL runs ($K=576$ reduction, $K_T=K/M=36$ K-tiles, $N_T=C_{out}/N=4$ N-tiles, $P=\text{batch} \cdot H_{out} \cdot W_{out}=499,200$ output pixels). One output pixel needs $K \cdot C_{out} = 36,864$ MACs, which a 256-MAC/cycle array could finish in ≈ 144 cycles. The shipped per-pixel schedule instead spends

$$N_T \left[\underbrace{K_T \cdot 23}_{\text{load}} + (K_T - 1) \cdot \underbrace{422}_{\text{compute_hold}} + \underbrace{438}_{\text{full}} \right] = 4 [36 \cdot 23 + 35 \cdot 422 + 438] = 64,144 \text{ cycles/pixel},$$

a utilization of only $144/64,144 \approx 0.22\%$. Summed over all pixels, the `compute_hold` term alone is

$$P N_T (K_T - 1) \cdot 422 = 2.95 \times 10^{10} \text{ cycles} = 295 \text{ s at } 100 \text{ MHz } (\mathbf{92\%} \text{ of the } 320.2 \text{ s}),$$

versus 16.5 s of weight loads and 8.7 s of drains. The 422-cycle hold is mostly the $(M+N-1) \cdot \text{MAC_LATENCY} = 155$ -cycle systolic pipeline-fill latency (plus FIFO/handshake overhead): a single 16-wide activation column produces only $M \cdot N = 256$ MACs, so the fill is never amortized. Weight-stationary pays off only when many activation columns stream through one resident weight load – the design streams exactly one, so this fill latency is re-paid for every pixel and every K-tile. That is the precise sense in which `compute_hold` scales with the pixel count, and it is what the streaming fix in Sec. 9 targets.

9 What did not work

Two related setbacks define this milestone: a design-process blindspot, and a failed attempt to fix it under a one-shot PnR constraint.

The blindspot (no pixel reuse). The shipped schedule pins one output pixel in the N-wide `result_buf` and reloads the full weight slab on every K-tile, reusing each weight set across exactly one pixel. New to the hardware/software co-design space, I made a process mistake: I assumed that if the compute datapath was correct, the interface bandwidth would simply be high enough for the system to run cleanly, so I skipped a full design integration. The flaw only surfaced when I ran a fully integrated benchmark and saw the per-tile costs extrapolate to RAFT scale (Sec. 8): AI collapses to 0.92 FLOP/byte, the array runs at 0.22% utilization, and the design ends up far slower than the CPU. The lesson is less about one design choice than about sequencing – I should have spent more time understanding the kernel and had an integration plan from the start, so the roofline/reuse analysis gated the dataflow *before* I invested in datapath timing closure.

The attempted fix (streaming GEMM). The remedy is to keep weights resident and stream a block of B pixel-columns through them, amortizing one weight load over B pixels. Modeling showed this would raise effective AI from ~ 0.92 to ~ 8.2 FLOP/byte ($\sim 9\times$) and move the operating point up the bandwidth roof (still memory-bound, but far faster). I implemented a register-based version (`act_block[B][M]`, `result_buf[B][N]`, a `PIX_BLOCK` parameter and runtime `cfg_pix_count`) keeping the $B=1$ path bit-exact; co-simulation passed.

Why it was reverted. The streaming netlist did not close timing, and only one PnR window remained. Pre-PnR (ideal clock) register-to-register setup worst slack was already -71.5 ns across ~ 30 k paths; post-CTS it settled at -10.4 ns on a 10 ns period (vs the shipped design’s -1.86 ns post-CTS, which the resizer then drove to met). The critical path ran from the pixel counter `compute_cycle[4]` (fanout 1,127) through a deep buffer tree and ~ 30 levels of index/decode logic

into the 32-deep array selection – the mux-cone pathology of indexing register arrays by a counter. The resizer could not converge and the run exhausted its window before routing, so I reverted to the verified, timing-closing design benchmarked here.

What I would do differently. The shortfall comes from three things: project scope, my experience with the tools, and my design process. For the bandwidth issue, the single highest-value change would have been a full system integration in M3 rather than the one-pass co-simulation I actually ran – that alone would have surfaced the reuse problem early enough to fix. For the streaming netlist, I would pipeline the address-decode (or use a shift-register activation feed) so the combinational cone fits the period, bound B to limit mux fanout, and budget more than one PnR iteration for any netlist-structural change. The remaining gap – peak compute below the CPU – traces to the 16×16 , 100 MHz scope I chose to keep synthesis tractable (Sec. 7); that was a conscious trade I stand by for a course project, not an oversight.

References / source artifacts

M1 software baseline: `project/m1/sw_baseline.md`, `interface_selection.md`.
M2 precision: `project/m2/precision.md` (`sweep tb/tb_quant_error.py`, `log sim/quant_error.log`).
RTL + verification: `project/m4/rtl/`, `tb/tb_top.py`, `sim/cosim_run.log`.
Synthesis: `project/m4/synth/` – `config.json`, `timing_report.txt`, `area_report.txt`, `power_report.txt`.
Benchmark: `project/m4/bench/` – `benchmark.md`, `benchmark_data.csv`, `roofline_final.png`.